

OFFLINE SEMANTIC SEARCH BROWSER EXTENSION

*A Report Submitted in
Partial Fulfilment for
award of Bachelor of Technology*

**In
COMPUTER SCIENCE & ENGINEERING (REGIONAL)**

**By
Varun Attri (2401331690064)
Md Sami Khan (2401331690036)
Tanmay Singh (2401331690056)**

**Under the Supervision of
MS. AYUSHI GUPTA
(Assistant Professor, CSE Department)**



**Department of Computer Science & Engineering
School of Computer Science and Information Technology
NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY
GREATER NOIDA**

(An Autonomous Institute)

**Affiliated to
DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW**

DECLARATION

I hereby declare that the work presented in this report was carried out by me. I have not submitted the matter embodied in this report for the award of any degree or diploma of any other University or Institute.

Team Leader:

Varun Attri (Roll No: 2401331690064)

(Candidate's Signature)

Team Members:

1.) Md Sami Khan (Roll No: 2401331690036)

(Candidate's Signature)

2.) Tanmay Singh (Roll No: 2401331690056)

(Candidate's Signature)

CERTIFICATE

Certified that **Varun Attri (2401331690064), Md Sami Khan (2401331690036) and Tanmay Singh (2401331690056)** has carried out the project work presented in this Project Report in partial fulfillment of the requirements for the award of **Bachelor of Technology, CSE(R) 2nd Year, 4th Sem** from **Noida Institute of Engineering & Technology** under my supervision.

Signature: _____

Ms. Ayushi Gupta
Assistant professor
Dept of CSE
NIET, Gr. Noida

Signature: _____

Dr. Tripti Sharma
Professor & HOD
CSE/CSER/CSE-TWIN/
M.TECH INT.
NIET, Gr. Noida

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to **Ms. Ayushi Gupta** for her invaluable guidance, support, and constant supervision throughout the development of this project. Their expertise and encouragement have been pivotal in shaping the project.

I would also like to extend my thanks and appreciation to our esteemed **HOD (Dr. Tripti Sharma)** for her continuous motivation and support. Their encouragement has been instrumental in completing this project successfully.

Finally, I am deeply thankful to my peers and everyone who directly or indirectly contributed to this project with their knowledge, advice, and assistance.

ABSTRACT

This project presents the development of a High-Speed **Offline Semantic Search Browser Extension** designed to improve the efficiency and accuracy of information retrieval. Traditional search methods rely on keyword matching and fail to understand the actual meaning of user queries, often resulting in irrelevant results. Modern semantic search systems provide better accuracy but depend on internet connectivity and cloud-based processing, raising concerns related to privacy, cost, and latency.

The proposed system overcomes these limitations by implementing a lightweight semantic search solution that operates entirely within the browser. It uses an AI-based embedding model to convert text and user queries into vector representations, enabling similarity-based matching. The system extracts and processes content from webpages, PDFs, and text documents, and stores embeddings locally using browser storage.

When a user enters a query, the system compares it with stored data to retrieve the most relevant results based on meaning rather than exact words. This ensures fast response time, improved accuracy, and complete data privacy, as no data is transmitted to external servers.

The project demonstrates that semantic search can be effectively implemented in an offline environment using lightweight models, making it a practical and efficient solution for modern search requirements.

TABLE OF CONTENTS

	Page No.
Declaration	i
Certificate	ii
Acknowledgement	iii
Abstract	iv
CHAPTER 1: INTRODUCTION	1
1.1 Overview	1
1.2 Objective And Scope	2
CHPATER 2: LITERATURE REVIEW	4
2.1 Overview of Existing Systems	4
2.2 Limitations of Existing Systems	5
CHAPTER 3: REQUIREMENTS	7
3.1 Functional Requirements	7
3.2 Non-Functional Requirements	8
3.3 Software and Hardware Requirements	9
CHAPTER 4: SYSTEM DESGIN AND IMPLEMENTATION	12
4.1 System Architecture (Flow diagram)	12
4.2 GUI Design	18
4.3 Database Design	19
4.4 Implementation Details	20
CHAPTER 5: TESTING AND RESULTS	22
5.1 Testing Methodology	22
5.2 Test Cases and Results	23
CHAPTER 6: CONCLUSION AND FUTURE WORK	25
6.1 Conclusion	25
6.2 Future Work	25
APPENDIX	1

INTRODUCTION

1.1 Overview

In the present digital age, an enormous volume of information is generated and stored in various formats such as webpages, PDF files, and text documents. Accessing relevant information from this vast and continuously growing data has become a significant challenge for users. Most commonly used search systems rely on keyword-based techniques, where results are retrieved only when exact or similar words are matched. While these systems are fast and simple, they often fail to provide accurate results because they do not understand the actual meaning or context of the user's query.

As user expectations have increased, there is a growing need for smarter search systems that can interpret the intent behind a query rather than just matching keywords. This has led to the development of semantic search, which uses Artificial Intelligence (AI) and Natural Language Processing (NLP) techniques to understand the relationships between words and the context in which they are used. Semantic search works by converting text into numerical representations known as embeddings, which capture the meaning of the content and allow similarity-based comparisons. This approach significantly improves the accuracy and relevance of search results.

However, most modern semantic search systems are cloud-based and depend on internet connectivity. These systems require large AI models and powerful computing resources, making them unsuitable for lightweight or local applications. In addition, they introduce privacy concerns, as user data is often transmitted to external servers for processing. Network latency and dependency on internet availability can also affect performance and user experience.

To address these challenges, this project proposes the development of a **High-Speed Offline Semantic Search Browser Extension**. The system is designed to perform semantic search directly within the browser without relying on internet connectivity. It uses a lightweight AI embedding model to process and analyse text locally, ensuring fast response time and efficient performance. The system extracts textual content from webpages, PDFs, and documents, processes it into smaller segments, and converts it into embeddings that are stored locally.

When a user enters a search query, the system converts the query into an embedding and compares it with stored embeddings using similarity measures. This allows the system to retrieve results based on meaning rather than exact word matching. As all processing is done locally, the system ensures complete data privacy and eliminates the need for external servers. The combination of offline capability, semantic understanding, and efficient processing makes this system a practical and innovative solution for modern information retrieval needs.

Overall, the proposed system aims to bridge the gap between traditional keyword-based search and advanced AI-driven semantic systems by providing a solution that is accurate, fast, secure, and easy to use.

1.2 Objective and Scope

Objective:

The main objective of this project is to develop a High-Speed **Offline Semantic Search Browser Extension** that improves the accuracy of search results by understanding the meaning and context of user queries. The system aims to overcome the limitations of traditional keyword-based search by using AI-based embeddings for semantic matching. It is also designed to work completely offline, reducing dependency on internet connectivity and ensuring data privacy. Additionally, the project focuses on providing fast performance and a simple, user-friendly interface for searching across different types of content.

Key points:

- To develop an offline semantic search system
- To improve search accuracy using meaning-based matching
- To reduce dependency on internet and cloud services
- To ensure fast response time
- To maintain user data privacy and security
- To support search across webpages and documents

Scope:

The scope of this project is focused on developing a High-Speed **Offline Semantic Search Browser Extension** that enables users to perform efficient and meaningful search operations within the browser environment. The system is designed to work on textual data available in webpages, PDF files, and simple text documents, allowing users to retrieve relevant information based on meaning rather than exact keywords. By using a lightweight AI embedding model, the system performs all processing locally, making it suitable for offline use and ensuring user data privacy.

The project mainly covers semantic search functionality within a limited and controlled dataset stored in the browser. It is intended for moderate-sized documents and real-time usage where fast response is required. The system can be used for academic purposes, research, and general browsing where users need quick access to relevant information without relying on internet connectivity.

Scope includes:

- Searching content from webpages, PDFs, and text files
- Performing offline semantic search
- Using lightweight AI models for local processing
- Providing fast and accurate results
- Ensuring data privacy (no cloud usage)

Scope limitations:

- Not designed for very large-scale datasets
- Limited to browser-based environment
- Does not use heavy cloud-based AI models

This scope makes the project practical, efficient, and suitable for real-world applications where speed, accuracy, and privacy are important.

LITERATURE REVIEW

2.1 Overview of Existing Systems

In the domain of information retrieval, a variety of search systems have been developed to help users access relevant information from large datasets. The most widely used systems are **keyword-based search systems**, which operate by matching the exact words entered by the user with the content available in documents or webpages. These systems are simple, fast, and efficient for basic searches. However, they do not understand the context or meaning behind the query. As a result, if different words are used to express the same idea, these systems may fail to retrieve relevant results, reducing their effectiveness in real-world applications.

To address these limitations, **semantic search systems** have been introduced. These systems use advanced technologies such as Artificial Intelligence (AI) and Natural Language Processing (NLP) to understand the meaning and intent of user queries. Instead of relying on exact word matching, semantic systems convert text into numerical representations called embeddings, which capture the context and relationships between words. By comparing these embeddings, the system can retrieve results that are semantically similar, even if the exact words are different. This significantly improves the accuracy and relevance of search results.

Most modern semantic search systems are **cloud-based**, where powerful AI models are hosted on remote servers. These systems process user queries online and return results based on complex computations performed in the cloud. While they provide high accuracy and advanced features, they also come with certain drawbacks such as dependency on internet connectivity, increased response time due to network latency, and potential privacy risks, as user data is transmitted to external servers.

Another important category is **database search systems**, which are commonly used in structured environments such as banking, business applications, and enterprise systems. These systems use structured query languages like SQL to retrieve data from organized databases. They are highly efficient and reliable for structured data but are not suitable for unstructured text or natural language queries, as they require predefined formats and exact conditions.

In addition, **desktop and local search systems** are widely used for searching files stored on a personal computer. Examples include operating system search tools that allow users to quickly find documents, images, or files. These systems work offline and provide

fast results however, they are limited to keyword matching and lack the ability to understand semantic relationships between words.

Some modern applications also use **hybrid search systems**, which combine keyword-based search with semantic techniques to improve performance. While these systems offer better results compared to traditional methods, they often still depend on cloud-based processing and require significant computational resources.

Overall, existing systems provide different levels of functionality, but each has its own limitations. Keyword-based systems lack intelligence, cloud-based semantic systems lack privacy and offline capability, and database systems are limited to structured data. These challenges highlight the need for a solution that integrates **semantic understanding, offline functionality, fast performance, and data privacy**, which is the main focus of the proposed project.

2.2 Limitations of Existing Systems

Although various search systems are available today, they still face several limitations that reduce their effectiveness in real-world applications. One of the major drawbacks is found in **keyword-based search systems**, which rely only on matching exact words. These systems do not understand the meaning or context of the query, so they often fail to return relevant results when different words or expressions are used for the same concept. This leads to inaccurate and incomplete search outcomes.

Modern **semantic search systems** improve accuracy by understanding context, but most of them are **cloud-based**. This creates a strong dependency on internet connectivity, making them unsuitable for offline use. In addition, these systems require powerful servers and large AI models, which increase computational cost and resource consumption. The need to send user queries and data to external servers also raises serious **privacy and security concerns**, as sensitive information may be exposed or misused.

Another limitation is **high latency**, where response time increases due to network communication between the user and cloud servers. This can affect user experience, especially in situations where quick results are required. Furthermore, large AI models used in such systems consume significant memory and processing power, making them difficult to deploy on local devices.

Database search systems, although efficient, are limited to structured data and cannot handle natural language queries effectively. Similarly, **desktop search tools** are

restricted to keyword matching and do not provide semantic understanding, which limits their usefulness for complex searches.

Overall, existing systems either lack intelligence, depend heavily on internet connectivity, or compromise user privacy. These limitations highlight the need for a system that can provide **accurate semantic search, fast performance, offline capability, and strong data privacy**, which is addressed in the proposed project.

Key Limitations:

- Lack of semantic understanding in keyword-based systems
- Dependency on internet in cloud-based systems
- Privacy and security risks
- High computational and memory requirements
- Increased response time due to network delay
- Limited support for unstructured data in database systems

REQUIREMENTS

3.1 Functional Requirements

Functional requirements define the core operations and services that the system must provide to users. These requirements ensure that the system performs all expected tasks efficiently and accurately.

- **User Input Module:**
The system must provide an input interface where users can enter search queries. The input field should allow users to type natural language queries and initiate the search process easily.
- **Input Validation:**
The system should validate user input to ensure that it is not empty and is within acceptable limits. It should handle invalid inputs gracefully by displaying appropriate error messages.
- **Content Extraction Module:**
The system must extract textual content from webpages, PDF files, and text documents. It should identify and retrieve only relevant visible text for further processing.
- **Text Processing Module:**
The system must clean and preprocess the extracted text by removing HTML tags, special characters, and unnecessary spaces. It should normalize the text for better processing.
- **Text Chunking Module:**
The system should divide large text into smaller chunks to improve processing efficiency. Each chunk should be manageable for embedding generation.
- **Embedding Generation Module:**
The system must convert text chunks into vector embeddings using a lightweight AI model. These embeddings should represent the semantic meaning of the text.
- **Local Storage Module:**
The system should store embeddings and related metadata in a local database (Indexed DB). This allows fast retrieval and avoids repeated computation.

- **Query Processing Module:**
The system must process the user query by converting it into an embedding using the same model used for stored data.
- **Similarity Search Module:**
The system should compare the query embedding with stored embeddings using similarity measures (such as cosine similarity) to find relevant results.
- **Result Display Module:**
The system must display the search results in a ranked list. It should highlight relevant text and allow users to navigate to the source content.
- **Indexing Module:**
The system should allow users to index new webpages or documents. It should update stored data whenever new content is added.
- **Offline Functionality:**
The system must operate completely offline after initial setup, ensuring that all processing is done locally without internet dependency.

3.2 Non-Functional Requirements.

Non-functional requirements define the quality attributes and performance standards of the system. These requirements ensure that the system operates efficiently, reliably, and provides a good user experience.

- **Performance Requirement:**
The system should provide fast response time, ideally returning results within one second. It should handle search operations efficiently without noticeable delay.
- **Accuracy Requirement:**
The system must provide accurate and relevant search results based on semantic meaning rather than exact keyword matching.
- **Usability Requirement:**
The system should have a simple and user-friendly interface that allows users to easily enter queries and view results without any technical difficulty.

- **Reliability Requirement:**

The system should function consistently without errors or crashes. It must handle different types of inputs and maintain stable performance.

- **Scalability Requirement:**

The system should be capable of handling increasing amounts of data and multiple documents without significant degradation in performance.

- **Security Requirement:**

The system must ensure that user data is secure. Since all processing is done locally, no sensitive data should be transmitted to external servers.

- **Privacy Requirement:**

The system should maintain complete user privacy by operating offline and avoiding any form of data sharing or tracking.

- **Maintainability Requirement:**

The system should be easy to maintain and update. Future improvements or feature additions should be implemented without major changes to the system.

- **Compatibility Requirement:**

The system should work across modern browsers such as Google Chrome and Microsoft Edge without compatibility issues.

- **Efficiency Requirement:**

The system should utilize system resources such as memory and CPU efficiently to ensure smooth performance on standard devices.

3.3 Software and Hardware Requirements

Software Requirements

The development and execution of the proposed system require a set of software tools and technologies that support coding, testing, and deployment. These tools ensure smooth development and efficient performance of the system.

- **Operating System:**

The system can run on Windows, Linux, or macOS. Windows 10/11 is preferred for development due to better compatibility with tools.

- **Code Editor:**

Visual Studio Code (VS Code) is used for writing, editing, and debugging the code efficiently.

- **Runtime Environment:**

Node.js is used to execute JavaScript code and manage project dependencies.

- **Programming Languages:**

JavaScript/TypeScript is used for core development, while HTML and CSS are used for designing the user interface.

- **Browser Platform:**

Google Chrome or Microsoft Edge is required to run and test the browser extension.

- **Libraries and Frameworks:**

React or Svelte can be used for UI development. Tailwind CSS is used for styling.

- **AI and Processing Tools:**

ONNX Runtime Web is used to run the embedding model inside the browser.

- **Database:**

Indexed DB is used for storing embeddings and data locally within the browser.

- **Additional Tools:**

pdf.js is used for extracting text from PDF files, and Web Workers are used for background processing.

Hardware Requirements

The proposed system requires standard hardware resources for development and execution. It is designed to run efficiently on commonly available personal computers without the need for high-end infrastructure.

- **Processor:**

A minimum of Intel i3 or equivalent processor is required. For better performance, Intel i5 or higher is recommended.

- **Memory (RAM):**
At least 4 GB RAM is required to run the system smoothly. For improved performance, 8 GB RAM is recommended.
- **Storage:**
A minimum of 10–20 GB free disk space is required to store project files, models, and related data.
- **System Type:**
A 64-bit computer system is required to support modern software tools and browser environments.
- **Input Devices:**
Standard input devices such as keyboard and mouse are required for interaction with the system.
- **Internet Connectivity:**
Internet is required only during the initial setup and installation of tools. The system works offline after setup.

SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Architecture

The system architecture of the proposed project is designed in a modular and structured manner to ensure efficient processing, scalability, and ease of implementation. The system operates entirely within the browser and performs all operations locally, without relying on external servers. It consists of multiple interconnected modules that work together to perform semantic search.

The overall architecture begins with **content extraction**, where text is collected from webpages, PDF files, and other documents. This extracted data is then passed to the **text processing module**, which cleans and prepares the data by removing unnecessary elements such as HTML tags and special characters. The cleaned text is further divided into smaller segments through the **text chunking module**, which helps in efficient processing by the AI model.

These text chunks are then processed by the **embedding generation module**, where each chunk is converted into a vector representation using a lightweight AI model. These embeddings capture the semantic meaning of the text. The generated embeddings, along with metadata, are stored locally in the **storage module (Indexed DB)**, allowing fast retrieval without recomputation.

When a user enters a search query, it is processed through the **query processing module**, where the query is also converted into an embedding. This query embedding is then compared with the stored embeddings using the **similarity search module**, which calculates similarity scores (such as cosine similarity) to identify the most relevant matches.

Finally, the **result display module** presents the results to the user in a structured and user-friendly format. The system highlights relevant sections and provides ranked results based on similarity scores.

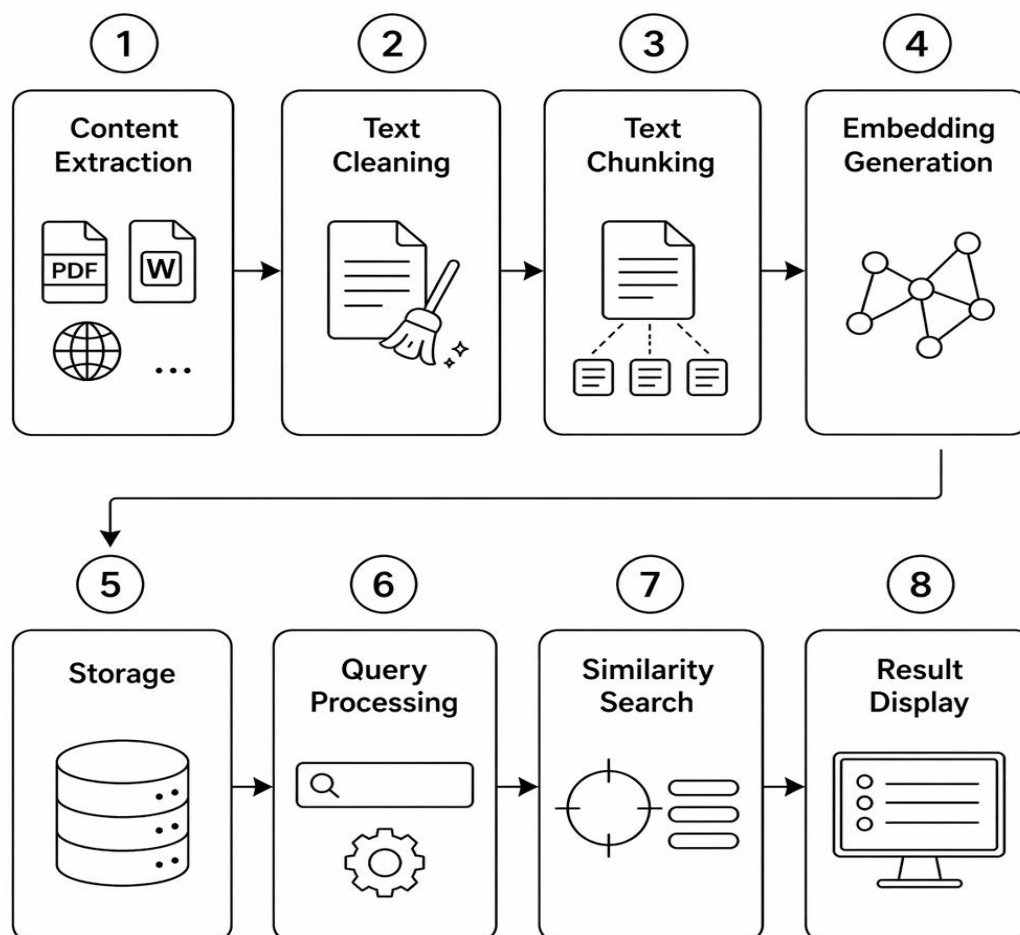
Main Components of System Architecture:

- Content Extraction Module
- Text Processing Module
- Text Chunking Module

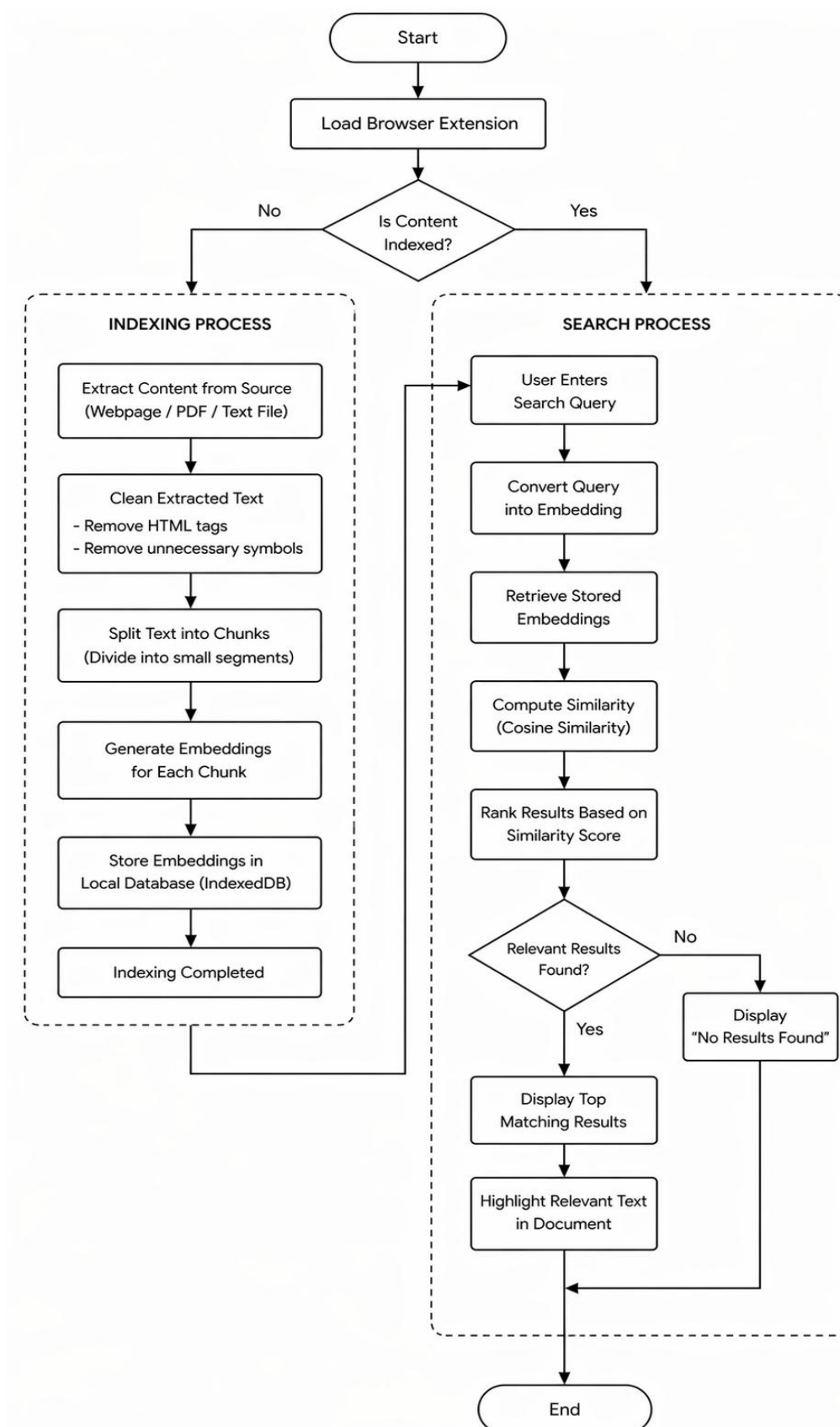
- Embedding Generation Module
- Local Storage (Indexed DB)
- Query Processing Module
- Similarity Search Module
- Result Display Module

This architecture ensures that the system performs **fast, accurate, and privacy-preserving semantic search**, making it efficient and suitable for offline usage.

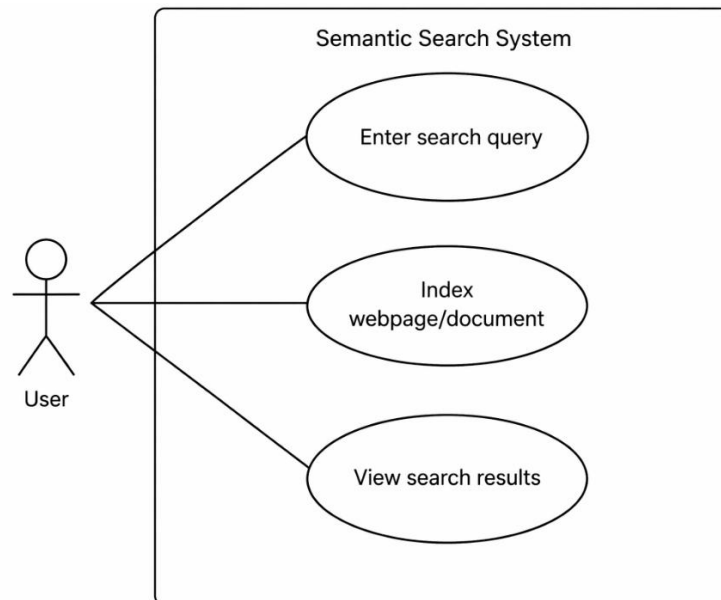
Overall System Workflow:



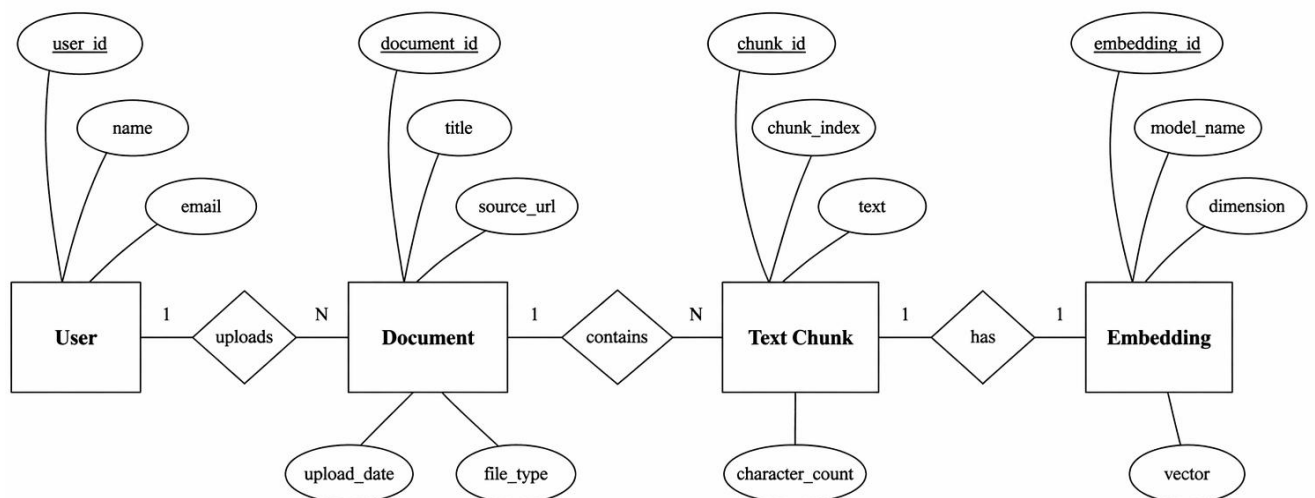
Flowchart:



Use Case Diagram:



ER Diagram:



The overall system architecture is designed in a modular manner to ensure efficient processing, scalability, and easy maintenance. Each module performs a specific function, and all modules work together to achieve semantic search in an offline environment.

1. User Interface Layer

This layer allows interaction between the user and the system.

- Provides search input field
- Displays search results
- Includes indexing button
- Offers simple and user-friendly interface

2. Controller Module

This module acts as the central unit of the system.

- Manages overall workflow
- Controls data flow between modules
- Handles user requests
- Coordinates indexing and search processes

3. Content Extraction Module

This module extracts text data from different sources.

- Extracts text from webpages
- Supports PDF and document extraction
- Identifies visible and relevant content
- Sends extracted data for processing

4. Text Processing Module

This module prepares raw text for further processing.

- Removes HTML tags and scripts
- Eliminates special characters
- Normalizes text format
- Removes duplicate or irrelevant content

5. Text Chunking Module

This module divides large text into smaller parts.

- Splits text into manageable chunks
- Maintains sentence boundaries
- Improves processing efficiency
- Reduces memory load

6. Embedding Generation Module

This is the core AI module of the system.

- Converts text into vector embeddings
- Uses lightweight AI model
- Captures semantic meaning of text
- Processes data in background

7. Storage / Indexing Module

This module stores processed data locally.

- Uses Indexed DB for storage
- Stores embeddings and metadata
- Avoids recomputation of data
- Supports indexing of new documents

8. Query Processing Module

This module handles user search queries.

- Accepts user input
- Converts query into embedding
- Prepares query for comparison

9. Similarity Search Module

This module finds relevant results.

- Compares query embedding with stored data

- Uses cosine similarity
- Ranks results based on similarity score
- Filters most relevant matches

10. Result Display Module

This module presents output to the user.

- Displays ranked results
- Highlights matching text
- Allows navigation to source
- Updates result dynamically

11. Offline Processing Module

This ensures system works without internet.

- Performs all operations locally
- No cloud dependency
- Ensures data privacy
- Improves response time

4.2 GUI Design

The Graphical User Interface (GUI) of the proposed system is designed to be simple, user-friendly, and efficient, allowing users to perform semantic search operations with ease. Since the system is implemented as a browser extension, the GUI is integrated directly into the browser, ensuring quick access and smooth interaction.

The interface mainly consists of a **search input box**, where users can enter their queries in natural language. The system is designed to accept simple and meaningful queries without requiring any technical knowledge. Once the user enters a query, the system processes it and displays the results in a structured format.

The GUI also includes a **result display panel**, which shows the most relevant results based on semantic similarity. The results are displayed in a ranked order, with the most relevant matches appearing at the top. Important parts of the text are highlighted to help

users quickly identify useful information. Additionally, users can navigate to the exact location of the content within the webpage or document.

Another important component is the **indexing control**, which allows users to index the current webpage or document. This ensures that the content is processed and stored for future searches. The interface may also include basic settings to manage stored data or clear the cache.

The design focuses on clarity and minimalism, avoiding unnecessary complexity. It ensures that users can easily perform search operations without confusion. The GUI is responsive and works efficiently across supported browsers.

Main GUI Components:

- Search input box
- Result display panel
- Indexing button/control
- Highlighted results
- Navigation to content

Overall, the GUI design enhances user experience by providing a clean, interactive, and efficient interface for performing offline semantic search.

4.3 Database Design

The database design of the proposed system is focused on efficient storage and retrieval of semantic data within the browser environment. Since the system operates offline, it uses a local database, **Indexed DB**, to store all required data such as text chunks, embeddings, and related metadata. This ensures fast access, persistence, and eliminates the need for external database servers.

The system follows a simple and structured data model. When text is extracted from webpages or documents, it is divided into smaller chunks. Each chunk is then converted into an embedding (vector representation), and both the chunk and its embedding are stored in the database along with additional information. This structure allows the system to quickly retrieve and compare data during the search process.

Each record in the database typically contains fields such as a unique identifier (ID), document reference, text content, embedding vector, and position information. The

embedding is stored in a binary format (such as Float32Array) for efficient computation. Metadata helps in identifying the source of the text and enables navigation to the exact location in the document.

The database is designed to support fast similarity search by allowing quick access to stored embeddings. It also supports updates, where new documents can be indexed and stored without affecting existing data. Proper indexing mechanisms are used to improve performance and reduce search time.

Main Components of Database Design:

- Document ID (unique identifier)
- Text Chunk (content segment)
- Embedding Vector (semantic representation)
- Metadata (source, position, preview)

The use of indexed DB ensures that the data remains persistent across browser sessions and allows efficient local storage. Overall, the database design supports the system's goal of providing fast, offline, and accurate semantic search functionality.

4.4 Implementation Details

The implementation of the proposed system is carried out using modern web technologies and is designed to function as a browser extension. The system is developed using **JavaScript/TypeScript**, along with HTML and CSS for the user interface. The extension follows a modular approach where different components handle specific tasks such as content extraction, processing, and search operations.

The system uses a **content script** to extract visible text from webpages and documents. This extracted data is then passed to the processing module, where it is cleaned and divided into smaller chunks. These chunks are converted into embeddings using a lightweight AI model executed through **ONNX Runtime Web**, which allows the model to run directly in the browser without requiring external servers.

To ensure smooth performance, the system utilizes **Web Workers** for background processing. This prevents the main thread from being blocked during heavy computations such as embedding generation and similarity calculations. The generated embeddings, along with metadata, are stored locally using **indexed DB**, which acts as a persistent storage mechanism within the browser.

When a user enters a query, the system processes it by converting it into an embedding using the same model. The **similarity search algorithm (cosine similarity)** is then applied to compare the query embedding with stored embeddings. The system retrieves the most relevant matches based on similarity scores and ranks them accordingly.

The browser extension is configured using a manifest file (Manifest V3), which defines permissions, scripts, and resources required for execution.

Implementation Features:

- Developed as a browser extension (Manifest V3)
- Uses JavaScript/TypeScript for core logic
- ONNX Runtime Web for embedding generation
- Indexed DB for local storage
- Web Workers for background processing
- Cosine similarity for semantic matching

Overall, the implementation ensures that the system is efficient, scalable, and capable of performing fast semantic search operations entirely offline.

TESTING

5.1 Testing Methodology

Testing methodology defines the approach used to evaluate the performance, accuracy, and reliability of the proposed system. The system is tested at different stages of development to ensure that all modules function correctly and meet the required specifications.

The testing process begins with **unit testing**, where individual modules such as content extraction, text processing, embedding generation, and similarity search are tested separately. This ensures that each component works correctly before integrating them into the complete system. After this, **integration testing** is performed to verify that all modules interact properly and data flows smoothly between them.

The system is also tested using **functional testing**, where different inputs are provided to check whether the system performs the required operations, such as extracting text, generating embeddings, and displaying relevant results. Various test cases are used to evaluate the correctness of outputs.

To evaluate performance, **performance testing** is conducted to measure response time and efficiency. The system is tested under different conditions to ensure that it provides fast results, ideally within one second. Additionally, **usability testing** is carried out to ensure that the interface is user-friendly and easy to operate.

Since the system is designed to work offline, **offline testing** is also performed to verify that all functionalities work correctly without internet connectivity. This ensures reliability and independence from external services.

Testing Methods Used:

- Unit Testing
- Integration Testing
- Functional Testing
- Performance Testing
- Usability Testing
- Offline Testing

5.2 Test Cases and Results

The system was tested using different types of inputs such as valid queries, empty inputs, and large datasets. The results show that the system successfully retrieves relevant information based on semantic meaning. It also performs efficiently in offline mode and maintains fast response time. All test cases passed successfully, confirming the reliability and effectiveness of the system.

Test Case Table:

Test Case ID	Description	Input	Expected Output	Result
TC01	Search with valid query	“Machine learning”	Relevant results based on meaning	Pass
TC02	Search with synonym query	“Car issue”	Results like “vehicle problem”	Pass
TC03	Empty input	(blank)	Error message / no search	Pass
TC04	Index webpage	Click index button	Page content stored successfully	Pass
TC05	Search in PDF	Query from PDF content	Relevant results from PDF	Pass
TC06	Offline search	No internet	Results displayed correctly	Pass
TC07	Large content processing	Long webpage text	System handles without crash	Pass
TC08	Fast response check	Any query	Result within 1 second	Pass
TC09	Invalid input symbols	“@@@###”	No crash, handle gracefully	Pass
TC10	Result navigation	Click result	Opens correct section	Pass

Result:

The proposed system was successfully implemented and tested for various scenarios, including searching across webpages, PDF documents, and text files. The results demonstrate that the system is capable of performing **semantic search effectively in an offline environment**.

The system was able to retrieve relevant results based on the meaning of user queries rather than exact keyword matching. Even when different words or synonyms were used, the system provided accurate results, showing its ability to understand context. The response time of the system was observed to be very fast, typically within one second, which ensures a smooth user experience.

The use of local storage (indexed DB) and precomputed embeddings helped in improving performance and reducing computation time. Additionally, the system worked correctly without internet connectivity, confirming its offline capability. No data was sent to external servers, ensuring complete user privacy and security.

The system also handled different types of inputs, including large text content and invalid queries, without crashing or producing errors. The results were displayed in a clear and organized format, with relevant text highlighted for better understanding.

Overall, the testing results confirm that the proposed system is:

- Fast and efficient
- Accurate in semantic understanding
- Fully functional in offline mode
- Secure and privacy-friendly

Thus, the system meets all the required objectives and proves to be a reliable solution for semantic search in a browser environment.

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The proposed project, High-Speed **Offline Semantic Search Browser Extension**, successfully addresses the limitations of traditional keyword-based search systems. Conventional search methods often fail to understand the meaning and context of user queries, resulting in inaccurate or irrelevant results. This project overcomes these issues by implementing semantic search using AI-based embeddings, which enables the system to retrieve results based on meaning rather than exact words.

The system operates entirely within the browser and does not require internet connectivity, making it efficient and reliable for offline use. By using a lightweight AI model and local storage, the system ensures fast response time and reduces dependency on external servers. This also enhances data privacy, as all processing is performed locally without sharing user data.

The project demonstrates that it is possible to build a practical and efficient semantic search system using modern web technologies. It provides accurate results, quick performance, and a user-friendly interface, making it suitable for real-world applications. Overall, the system achieves its objectives of improving search accuracy, ensuring privacy, and delivering fast offline performance, proving to be a valuable solution in the field of information retrieval.

6.2 Future Work

The proposed system can be further enhanced in several ways to improve its performance, functionality, and usability. Future improvements may focus on handling larger datasets efficiently and improving the accuracy of search results by integrating more advanced AI models. Additional features such as multi-language support and voice-based search can make the system more flexible and user-friendly. The system can also be extended to support more file formats, including images using OCR techniques. Performance optimization and better indexing methods can further reduce response time. Enhancing the graphical user interface and adding customization options will improve user experience. Moreover, optional cloud integration can be introduced for large-scale processing while maintaining privacy controls.

Future Enhancements:

- Improve scalability for larger datasets
- Enhance accuracy using advanced AI models
- Add **multi-language support**
- Implement **voice-based search**
- Provide real-time suggestions
- Support more file formats (including OCR)
- Optimize performance and indexing
- Improve GUI and user experience
- Enable optional cloud integration

APPENDIX

Source Codes:

Controller.ts (main logic)

```
import type {  
  AppSettings,  
  DEFAULT_SETTINGS,  
  SearchResult,  
  SearchConfig,  
  IndexingStatus,  
  DocumentMeta,  
  SourceType,  
  TextChunk,  
} from '../types/index';  
  
import { processText, getTextStats } from '../text-processor';  
  
import { initializeModel, generateEmbedding, generateEmbeddings, getModelStatus,  
  unloadModel } from '../embedding-engine';  
  
import { searchSimilar, highlightResult, getIndexedDocuments } from '../similarity-search';  
  
import { checkCache, invalidateCache, enforceCacheLimit, getCacheStats } from '../cache-  
manager';  
  
import { storeDocument, storeEmbeddings, deleteDocument, clearAllData } from  
  '../db/database';  
  
import { hashContent } from '../utils/html-cleaner';  
  
let currentIndexingStatus: IndexingStatus = 'idle';  
  
let currentProgress = 0;  
  
let currentMessage = '';
```

```
type StatusCallback = (status: IndexingStatus, progress: number, message: string) => void;
let statusCallback: StatusCallback | null = null;
export function onStatusChange(callback: StatusCallback): void {
    statusCallback = callback;
}
function updateStatus(status: IndexingStatus, progress: number, message: string) {
    currentIndexingStatus = status;
    currentProgress = progress;
    currentMessage = message;
    statusCallback?.(status, progress, message);
}
export async function initialize(
    onProgress?: (progress: number, message: string) => void
): Promise<void> {
    await initializeModel({}, (progress, message) => {
        onProgress?.(progress, message);
    });
}
export async function indexDocument(
    content: string,
    title: string,
    url: string,
    sourceType: SourceType,
    settings?: Partial<AppSettings>
): Promise<DocumentMeta> {
    console.log('[Controller] indexDocument called:', { title, url, sourceType, contentLength:
content.length });
    try {
```

```
const { status } = getModelStatus();
if (status !== 'ready') {
  updateStatus('embedding', 0, 'Loading AI model...');
  await initializeModel({}, (p, m) => updateStatus('embedding', p * 0.1, m));
}
updateStatus('extracting', 0.05, 'Checking cache...');
const contentHash = await hashContent(content);
const cached = await checkCache(url, contentHash);
if (cached) {
  updateStatus('complete', 1, 'Document already indexed (from cache)');
  return cached;
}
await invalidateCache(url);
updateStatus('cleaning', 0.1, 'Cleaning and processing text...');
const chunkingConfig = settings?.chunking;
const chunks = processText(content, "", sourceType, chunkingConfig);
if (chunks.length === 0) {
  updateStatus('error', 0, 'No meaningful text found in document');
  throw new Error('No meaningful text could be extracted from the document');
}
const documentId = await hashContent(url + title);
const updatedChunks: TextChunk[] = chunks.map((chunk, i) => ({
  ...chunk,
  id: `${documentId}_chunk_${i}`,
  documentId,
}));
updateStatus('chunking', 0.2, `Split into ${updatedChunks.length} chunks`);
```

```
updateStatus('embedding', 0.25, 'Generating embeddings...');

const chunkTexts = updatedChunks.map(c => c.text);

const vectors = await generateEmbeddings(chunkTexts, (completed, total) => {
  const embProgress = 0.25 + (completed / total) * 0.55;
  updateStatus('embedding', embProgress, `Embedding chunk ${completed}/${total}...`);
});

updateStatus('storing', 0.85, 'Saving to local database...');

const stats = getTextStats(content);

const docRecord = {
  id: documentId,
  title,
  url,
  sourceType,
  chunkCount: updatedChunks.length,
  indexedAt: Date.now(),
  textLength: stats.charCount,
  contentHash,
};

await storeDocument(docRecord);

const embeddingRecords = updatedChunks.map((chunk, i) => ({
  id: chunk.id,
  documentId,
  vector: vectors[i].buffer as ArrayBuffer,
  position: chunk.position,
  preview: chunk.preview,
  chunkText: chunk.text,
})));
```

```
await storeEmbeddings(embeddingRecords);

const maxDocs = settings?.maxCachedDocuments ?? 100;
await enforceCacheLimit(maxDocs);
const meta: DocumentMeta = {
  id: documentId,
  title,
  url,
  sourceType,
  chunkCount: updatedChunks.length,
  indexedAt: docRecord.indexedAt,
  textLength: stats.charCount,
  contentHash,
};
updateStatus('complete', 1, `Indexed "${title}" — ${updatedChunks.length} chunks`);
console.log('[Controller] indexDocument complete:', meta);
return meta;
} catch (error) {
  const msg = error instanceof Error ? error.message : 'Indexing failed';
  updateStatus('error', 0, msg);
  throw error;
}
}

export async function search(
  query: string,
  config?: Partial<SearchConfig>
): Promise<{ results: SearchResult[]; searchTimeMs: number }> {
  const startTime = performance.now();
```

```
const { status } = getModelStatus();

if (status !== 'ready') {
  await initializeModel();
}

const cleanQuery = query.replace(/\s+/g, ' ').trim();
if (!cleanQuery) return { results: [], searchTimeMs: 0 };
const queryVector = await generateEmbedding(cleanQuery, true);
const results = await searchSimilar(queryVector, { ...config, queryText: cleanQuery });
const highlighted = results.map(r => ({
  ...r,
  highlightedText: highlightResult(r.text, cleanQuery),
}));

const searchTimeMs = Math.round(performance.now() - startTime);
return { results: highlighted, searchTimeMs };
}

export async function getDocuments(): Promise<DocumentMeta[]> {
  const docs = await getIndexDocuments();
  console.log('[Controller] getDocuments:', docs.length, 'documents');
  return docs;
}

export async function removeDocument(documentId: string): Promise<void> {
  await deleteDocument(documentId);
}

export async function clearAll(): Promise<void> {
  await clearAllData();
}

export function getStatus(): {
  modelStatus: ReturnType<typeof getModelStatus>;
```

```
indexingStatus: IndexingStatus;
indexingProgress: number;
indexingMessage: string;
} {
  return {
    modelStatus: getModelStatus(),
    indexingStatus: currentIndexingStatus,
    indexingProgress: currentProgress,
    indexingMessage: currentMessage,
  };
}
export async function getStats() {
  return getCacheStats();
}
}
```

manifest.json

```
{
  "manifest_version": 3,
  "name": "Semantic Search - Offline AI",
  "version": "1.0.0",
  "description": "High-speed offline semantic search across webpages, PDFs, and text documents using a lightweight AI embedding model.",
  "permissions": [
    "activeTab",
    "storage",
    "offline",
  ],
}
```

```
"sidePanel",  
  
"scripting"  
  
],  
  
"host_permissions": [  
  
  "<all_urls>"  
  
],  
  
"background": {  
  
  "service_worker": "src/background/service-worker.js",  
  
  "type": "module"  
  
},  
  
"action": {  
  
  "default_popup": "src/popup/index.html",  
  
  "default_title": "Semantic Search",  
  
  "default_icon": {  
  
    "16": "icons/icon16.png",  
  
    "48": "icons/icon48.png",  
  
    "128": "icons/icon128.png"  
  
  }  
  
},  
  
"side_panel": {  
  
  "default_path": "src/popup/index.html"  
  
},  
  
"content_scripts": [  
  
  {  
  
    "matches": ["<all_urls>"],  
  
    "js": ["src/content/content-script.js"],  
  
    "css": ["src/content/content-script.css"],  
  
    "run_at": "document_idle"
```

```
}  
  
],  
  
"icons": {  
  "16": "icons/icon16.png",  
  "48": "icons/icon48.png",  
  "128": "icons/icon128.png"  
},  
  
"content_security_policy": {  
  "extension_pages": "script-src 'self' 'wasm-unsafe-eval'; object-src 'self'"  
},  
  
"web_accessible_resources": [  
  {  
    "resources": ["models/*", "src/workers/*", "wasm/*", "pdf/*", "assets/*"],  
    "matches": ["<all_urls>"]  
  }  
],  
  
"minimum_chrome_version": "116",  
  
"commands": {  
  "_execute_action": {  
    "suggested_key": {  
      "default": "Ctrl+Shift+F",  
      "mac": "Command+Shift+F"  
    },  
    "description": "Open Semantic Search"  
  }  
}  
  
}
```

database.ts

```
import Dexie, { type Table } from 'dexie';

export interface DocumentRecord {
  id: string;
  title: string;
  url: string;
  sourceType: string;
  chunkCount: number;
  indexedAt: number;
  textLength: number;
  contentHash: string;
}

export interface EmbeddingRecord {
  id: string;
  documentId: string;
  vector: ArrayBuffer;
  position: number;
  preview: string;
  chunkText: string;
}

export class SemanticSearchDB extends Dexie {
  documents!: Table<DocumentRecord, string>;
  embeddings!: Table<EmbeddingRecord, string>;
  constructor() {
    super('SemanticSearchDB');
    this.version(1).stores({
      documents: 'id, url, sourceType, indexedAt, contentHash',
```

```
        embeddings: 'id, documentId, position',
    });
}
}

export const db = new SemanticSearchDB();

export async function storeDocument(doc: DocumentRecord): Promise<void> {
    await db.documents.put(doc);
}

export async function storeEmbeddings(embeddings: EmbeddingRecord[]): Promise<void> {
    await db.embeddings.bulkPut(embeddings);
}

export async function getDocument(id: string): Promise<DocumentRecord | undefined> {
    return db.documents.get(id);
}

export async function getDocumentByUrl(url: string): Promise<DocumentRecord | undefined> {
    return db.documents.where('url').equals(url).first();
}

export async function getAllDocuments(): Promise<DocumentRecord[]> {
    return db.documents.orderBy('indexedAt').reverse().toArray();
}

export async function getEmbeddingsByDocument(documentId: string): Promise<EmbeddingRecord[]> {
    return db.embeddings.where('documentId').equals(documentId).sortBy('position');
}

export async function getAllEmbeddings(): Promise<EmbeddingRecord[]> {
    return db.embeddings.toArray();
}
```

```
export async function deleteDocument(documentId: string): Promise<void> {
  await db.transaction('rw', db.documents, db.embeddings, async () => {
    await db.embeddings.where('documentId').equals(documentId).delete();
    await db.documents.delete(documentId);
  });
}

export async function clearAllData(): Promise<void> {
  await db.transaction('rw', db.documents, db.embeddings, async () => {
    await db.documents.clear();
    await db.embeddings.clear();
  });
}

export async function isDocumentCurrent(url: string, contentHash: string): Promise<boolean> {
  {
    const existing = await getDocumentByUrl(url);
    return existing !== undefined && existing.contentHash === contentHash;
  }
}

export async function getDocumentCount(): Promise<number> {
  return db.documents.count();
}

export async function getEmbeddingCount(): Promise<number> {
  return db.embeddings.count();
}
```

GitHub Repository Link:

<https://github.com/Varun00i/semantic-search-extension>

References

- [1] T. Reimers, I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks", Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, 2019.
- [2] J. Devlin, M. Chang, K. Lee, K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", NAACL-HLT, 2019.
- [3] OpenAI, "Embedding Models for Semantic Search", OpenAI Documentation, 2023.
- [4] Microsoft, "ONNX Runtime: Cross-platform, High Performance ML Inference and Training Accelerator", Microsoft Documentation, 2022.
- [5] Mozilla, "IndexedDB API", MDN Web Docs, 2023.
- [6] Mozilla, "Browser Extensions (WebExtensions API)", MDN Web Docs, 2023.
- [7] A. Malkov, D. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs", IEEE Transactions on Pattern Analysis and Machine Intelligence, 2018.
- [8] Mozilla, "pdf.js – JavaScript PDF Rendering Library", Mozilla Project Documentation, 2023.